Experiment: Basic SQL Injection on a Local Web Server

Aim

To set up a simple local web application and understand how an insecure SQL query can be manipulated using SQL injection.

Requirements

- Python 3
- Web browser
- No Internet required

Step 1: Create a folder

Create a folder:

```
sql_injection_lab
```

Inside it, create a file:

```
app.py
```

Step 2: Use this simple vulnerable application

```
from flask import Flask, request, render_template_string
import sqlite3

app = Flask(__name__)

HTML = """
<h2>Student Login System</h2>

<form method="POST">
    Username: <input type="text" name="username"><br><br>
    Password: <input type="password" name="password"><br><br>
    <input type="submit" value="Login">
</form>

<p>{{ message }}</p>
"""

def init_db():
    conn = sqlite3.connect("students.db")
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            username TEXT,
            password TEXT
```

```
        )
    """)

    cursor.execute("DELETE FROM users")

    cursor.execute(
        "INSERT INTO users VALUES ('admin', 'admin123')"
    )

    cursor.execute(
        "INSERT INTO users VALUES ('student', 'student123')"
    )

    conn.commit()
    conn.close()


@app.route("/", methods=["GET", "POST"])
def login():

    message = ""

    if request.method == "POST":

        username = request.form["username"]
        password = request.form["password"]

        conn = sqlite3.connect("students.db")
        cursor = conn.cursor()

        # INTENTIONALLY VULNERABLE QUERY - ONLY FOR LAB
        query = (
            "SELECT * FROM users WHERE username='"
            + username +
            "' AND password='"
            + password +
            "'"
        )

        print("SQL Query:", query)

        cursor.execute(query)
        result = cursor.fetchone()

        conn.close()

        if result:
            message = "Login Successful!"
        else:
            message = "Invalid Username or Password"

    return render_template_string(HTML, message=message)


if __name__ == "__main__":
    init_db()
    app.run(debug=True)
```

Step 3: Install Flask

Open Terminal or Command Prompt inside the folder and run:

```
pip install flask
```

Then run:

```
python app.py
```

You should see something similar to:

```
Running on http://127.0.0.1:5000
```

Open:

```
http://127.0.0.1:5000
```

Step 4: First demonstrate normal login

Tell students to enter:

```
Username: admin
Password: admin123
```

Result:

```
Login Successful!
```

Now try:

```
Username: admin
Password: wrongpassword
```

Result:

```
Invalid Username or Password
```

Step 5: Explain the vulnerable SQL query

The application creates a query like:

```
SELECT * FROM users
WHERE username='admin'
AND password='admin123'
```

Explain to students:

"The problem is that the application directly joins user input into the SQL query."

If the user enters specially crafted input, the structure of the SQL query may change.

Safe classroom demonstration

Use a simple example in the **local lab only** to show how an input containing an SQL condition and a comment marker can alter the intended `WHERE` clause. The key teaching point is to **print the generated SQL query** and let students compare it with the normal query, rather than practicing against any real website or system.

Ask students:

1. What was the original SQL query?
2. What query was generated after the special input?
3. Which part of the authentication condition changed?
4. Why did direct string concatenation create the vulnerability?

Step 6: Show the secure solution

After demonstrating the vulnerable concept, replace:

```
query = (
    "SELECT * FROM users WHERE username='"
    + username +
    "' AND password='"
    + password +
    "'"
)

cursor.execute(query)
```

with a **parameterized query**:

```
query = """
SELECT * FROM users
WHERE username=? AND password=?
"""

cursor.execute(query, (username, password))
```

Explain

Vulnerable:

```
User Input → Directly added to SQL query → Query can be altered
```

Secure:

```
User Input → Parameterized Query → Input treated as data
```